Handoff 15: Lesson feedback engine, part 1 (ENG-04)

Date: 2026-10-27 (America/Toronto)
Written by: Claude Code (implementer)
For: the human maintainer, and ChatGPT or Claude chat (reviewer) without repository access
Status: **in_review**: one pull request (PR_NUMBER), not merged, not deployed. **No database migration. No new dependency. No content change.**

> **How to use this file:** upload it with `build.md`, `docs/plan/lesson-feedback.md` (the specification), `docs/plan/browser-checks.md` and `docs/plan/teaching-design.md`. The pull request diff is the source of truth (`build.md` section 14).

Starter prompt for the chat assistant

```
You are reviewing Python Academy stage 15 (ENG-04, part 1): the lesson
feedback engine. When a learner's Python program fails, the page now
explains the error in plain words (19 translations from
lesson-feedback.md section 4), highlights the failing line in the editor,
and keeps Python's own message one click away. Results are labelled by
state, only a "passed" check counts as a pass, and hints are a three-step
ladder beside the code with no answer button. I'm attaching build.md,
docs/plan/lesson-feedback.md and this handoff. Check every message
against the specification, check that no message or hint gives away a
full solution, check that a run which only printed something can never
look like a pass, and question where I split the work. Explain at a
first-year computer engineering level.
```

1. Where we are

Step 0 (checks before building), 2026-10-27:

Check	Result
`git fetch --all`, `gh pr list --state all`	Done
PR #17 (ENG-03 browser behaviour checks) merged into `main`?	**Yes**, merged 2026-10-27 04:27 UTC (merge commit `4a06765`)

Check	Result
Branch base	`origin/main` at `7eab930` (after PR #18). Not stacked on any open branch
Earlier ENG-04 work on any branch, stash or PR	**None.** Only the specification (`lesson-feedback.md`, PR #15) and backlog rows mention it

Stage	What	Status
11	ENG-03 browser behaviour checks	Merged (PR #17)
12	Admin and tester roles (FB-01)	PR #21, open
13	Site design direction (SITE-01)	PR #19, open
14	Decision log and book outline	Merged (PR #18)
15	**ENG-04 lesson feedback, part 1**	**in_review: this PR**
16	Backlog sync	PR #20, open

2. Handoff record (build.md template)

Field	Value
Task	ENG-04: friendly translations of the 19 listed Python errors with the raw message one click away; the failing line highlighted without a new dependency; a three-level hint ladder beside the error that never reveals a full solution; honest results that work with ENG-03, using the existing result states
Human goal	A beginner who hits an error knows what happened, where, and one next step, and a pass always means the check passed
Relevant sections	`build.md` sections 4 (teaching contract), 11 and 12; `lesson-feedback.md` sections 4, 5, 7 and 8; `browser-checks.md` section 4
Repository and branch	`torontojohn/python-academy`, branch `feat/lesson-feedback`
Base commit / head	Base `7eab930`. Code and tests in `cb069a5`; documents in the commit after it
Implementer / reviewer	Claude Code / the owner and a chat assistant
Non-goals	Named check rules and their messages (section 6), hints tagged to errors (section 7), predict-first (section 9), measurements (section 10), unit 2 content, calling the ENG-03 runner from the page (UI-05), any grading change
Data migration / compatibility	None. The API is unchanged. Learners' saved work, XP and badges are untouched

Field	Value
Deployment status	Not requested

Files changed

File	Why
`frontend/app/feedback.ts` (new)	The engine, as pure functions: reads Python's error text, picks the translation, finds the line, cleans Python's message, labels result states, names the hint levels
`frontend/app/code-editor.tsx` (new)	The textarea with a line-number gutter and a highlight layer behind it, and "Go to line"
`frontend/app/adventure.tsx`	Uses both: shows results by state, explains errors, highlights the line, puts the hint ladder beside the code. Only `result: "passed"` is a pass
`frontend/app/style.css`	Styles for the editor layers, the result states and the hints. Classes only, no inline styles
`frontend/public/python-worker.js`	Unit 1 mode now reports the error type, keeps the output printed before an error, says when the output limit stopped the program, runs the reply helper as `<academy>`, and removes network access, as check cases already did
`frontend/tests/feedback.spec.ts` (new)	44 tests: unit tests of the engine on real Pyodide errors, and browser tests of the page
`frontend/tests/fixtures/pyodide-errors.json` (new)	22 real error texts from the Pyodide 0.27.7 worker, one or more per translation
`frontend/tests/fixtures/translations.snapshot.json` (new)	Every message template, so any change to one shows up in review
`frontend/tests/fixtures/hello-hi-woah.json` (new)	The real API answer for `print("hi")` then `print("woah")`
`frontend/tests/academy.spec.ts`	Updated for the new hint and error wording; the mocked check now returns a result state, as the real API does
`backend/tests/test_honest_checks.py`	One test: the `hi/woah` fixture is exactly what `POST /api/check` returns
`docs/plan/lesson-feedback.md`, `docs/plan/browser-checks.md`	Status lines: what part 1 built, what is left

File	Why
`docs/plan/decision-log.md`	Chapter 14 for this stage
`docs/handoffs/2026-10-27-15-lesson-feedback.md` and `.pdf`	This handoff

`docs/plan/backlog.md` is **not** edited (section 11 lists the rows to add).

3. How it works

1. The learner runs their code. The worker runs it in Pyodide, in their browser, as before.
2. If Python stops with an error, the worker sends the error text, the error type and what was printed before the error.
3. `feedback.ts` reads the line from the **last** `File "<exec>"` frame. `<exec>` is the learner's code; Pyodide's own frames are ignored. It then picks the first translation in the specification's order whose pattern matches.
4. The page shows: - the state ("Python stopped") - the title - where to look ("Line 3", or "On or just before line 2" for a syntax error) - the explanation - a **Go to line** button - **Show Python's message**, which is Python's own traceback without Pyodide's frames
5. The line gets a ▶ marker in the gutter and a tinted background. It clears when the learner changes that line or runs again.
6. On the challenge card, a program that ran without an error is sent to the check as before. The result is shown by its state. **Only** `result: "passed"` **unlocks the badge.**
7. On the "Try it" card there is no check, so a run shows **"Ran · not checked"**, never a pass.

Nothing new leaves the browser. The code and Python's message go nowhere new. A program that crashes is still not sent to the check.

Before and after

Situation	Before	After
`print(heor)` when `hero` exists	The raw traceback, including Pyodide's internal frames, then "Check the variable or function name…"	"Python does not know `heor`", "Line 3", "Did you mean `hero`?", line 3 marked, Python's message one click away
Missing `:` after `if`	The raw `SyntaxError` and a general tip	"This line needs a colon", "On or just before line 2"
Output before an error	Replaced by the error	Stays in the Output panel
Printing 20,000+ characters	`OSError: [Errno 29] I/O error`	"Too much output"
Run on "Try it"	"Your code ran…" in the same green box as a pass	"Ran · not checked", in a neutral box

Situation	Before	After
Check says `passed: true` but another state	Badge shown	No badge; the state decides
Hints	"Show next hint", "Hint 1." in the left panel	"Show a nudge / a bigger hint / the approach" beside the code; then "That is every hint", and no answer button

4. Where I split the work

The specification has five parts. This PR builds the three the task names, and leaves the rest for one more stage, because those need **content changes** that deserve their own review.

Part	This PR	Proposed next stage (ENG-04 part 2)
1. Error translations (section 4)	**Built**, all 19	—
2. Line highlight (section 5)	**Built**	—
3. Check-specific messages (section 6)	Results shown by state; ENG-03 `failed.line` highlighted	Named rules and message IDs in the content files (`hello.two-lines`, …)
4. Hints beside the error (section 7)	**Ladder built**, beside the code, one level at a time, no answer button	Hints **tagged** to error types or rules, so a `NameError` offers the spelling hint first; "a similar example" after hint 3; "Other ways to do it" after passing; optional highlighting of the hint button after two failed runs (open decision 3)
5. Predict-first (section 9)	Not built	Content flags per card, the prediction step, matched/not-matched support data
ENG-03 on the page	Results from the judge are shown correctly (tested with its real response shapes)	**With UI-05**: the page only loads unit 1, which has no browser checks, so calling `check-runner.js` from the page waits until unit 2 lessons can be loaded
Measurements (section 10)	Not built	"Same error on the next run" and "hint 3 shown" as support data, with no code or message text

5. Decisions and why

Decision	Options considered	Reason	Who decided
`print("hi")` **then** `print("woah")` **still passes**; the browser test checks the pass is honest and that a "Try it" run never looks like a pass	Add a word minimum so it fails, as the task's test list first asked	It conflicted with decision 5 (no word minimum, built by ASSESS-07) and with the specification's "no grading changes". I asked; the owner chose to keep decision 5	Owner, 2026-10-27
Split as in section 4	Everything in one PR	Rule IDs and hint tags change the content format and every lesson file	Implementer
Gutter and highlight layer behind the plain `<textarea>`	An editor library (CodeMirror)	No new dependency; the specification proposes this (section 11, decision 1)	Specification; implementer built it
Editor lines **no longer wrap**; long lines scroll sideways inside the editor	Keep wrapping	With wrapping, a long line takes two rows, and the highlight would land on the wrong row	Implementer
The highlight layers copy the textarea's scroll position from script	React `style` props	`style="…"` attributes are blocked once the page policy is enforced; setting `scrollTop` from script is not a style attribute	Implementer
Only `result === "passed"` (with `passed: true`) is a pass; an unknown state shows "Check unavailable"	Trusting `passed` alone	One source of truth for a pass	Implementer
Unit 2 translations (`+` with a number, `int()` of text, division by zero) give the general message in unit 1	Use them everywhere	The specification: "use the Anything else message instead of teaching ahead"	Specification
The timeout and output-limit messages are used in every unit	Hold the timeout back to unit 3, as the table's "(Unit 3)" note could mean	They describe what the app did, not Python, and there is no line for the general message to name	Implementer (**assumption 1**)
A missing block after `else` or `elif` uses the indent translation, with that word in the title	Only after `if`, as the table's example	The explanation already says "an `if` or `else`"	Implementer (**assumption 2**)
"Python knows `{known}`, not `{name}`" names the real built-in	The table's fixed "`print`"	`Input("Name?")` should not say "Python knows `print`"	Implementer (**assumption 3**)
"Words need quotes" only when the reported line has no quotes at all	Every "Perhaps you forgot a comma?"	With quotes on the line, a comma really may be missing; the table says "with words but no quotes"	Specification

Decision	Options considered	Reason	Who decided
Name suggestions: a name the learner defined, one or two edits away; else a built-in only if Python suggested it and learners know it	Always Python's suggestion	Python suggests `help` for `hello`, a name no learner has met	Implementer
The friendly message comes first in every unit	Python's message first from unit 6	Only unit 1 exists; open decision 2 in the specification	Specification default
The last hint button stays, marked `aria-disabled`	Remove or disable it, as before	Removing or disabling a focused button loses keyboard focus	Implementer
Unit 1 mode now also removes the network APIs	Only check cases	The task: "No network access from learner code"	Task rule

6. Bugs found and how they were fixed

Bug or near-miss	How it was found	Fix, and the test that guards it
Too much output looked like a disk error. In unit 1 mode the stop came back as `OSError: [Errno 29] I/O error`, so no message could recognise it	Capturing real Pyodide errors for the test fixtures	The worker sends `outputLimit: true`. Guarded by "the output limit is known from the worker flag"
A missing reply pointed inside our own helper (unit 1 mode). ENG-03 fixed this for check cases only	Reading the worker	The helper runs as `<academy>` in both modes. Guarded by `translation: replies` (line 2, not line 9)
Output printed before an error was hidden	Building the page	The output stays. Guarded by "an error inside a function…" (checks "A forest." is still shown)
Keyboard focus was lost after the last hint (also true before this PR)	My review of my own diff	`aria-disabled` button. Guarded by the hint ladder test (focus stays on it)
Near-miss: two of my own tests were wrong (an edit that also created a new error; a click on an `aria-disabled` button, which Playwright waits on forever)	The tests failed	Fixed the tests, not the code
Near-miss: a server left running from screenshots made Playwright stop before running anything; the filtered output was empty	I checked the empty result instead of reading it as a pass	Stopped the server and reran

7. How it was tested

Commands and results (on `cb069a5`, macOS, Node 24, Python 3.12.14)

Command	Result
`cd backend && DATABASE_URL=sqlite:///<temp>.db .venv/bin/python -m pytest -q`	**256 passed, 9 skipped**
Same with `TEST_POSTGRES_URL=<local PostgreSQL 16 from pgserver>`	**265 passed**
`cd frontend && npx tsc --noEmit`	OK
`cd frontend && npm run build`	OK
`cd frontend && CI=1 npx playwright test`	**83 passed** (39 before this PR, plus 44 new)
axe-core 4.10.3 (WCAG 2.0/2.1/2.2 A and AA plus best practices) on 6 states × desktop and phone (error with Python's message open, syntax error, honest pass, needs work, all three hints, "Try it" run)	**0 violations**; no sideways scrolling at 375 px

`DATABASE_URL` points at a throwaway file because a local, git-ignored `academy.db` from the admin branch is already at migration 0007. That only affects this Mac; CI has no such file.

What the tests cover

Kind of test	What it proves	What it does not prove
Unit tests on 22 real Pyodide error texts	Each of the 19 translations: title, line and "where" (including a syntax error found on line 2, an unclosed `(` on the line that opened it, and an error inside a function)	That a learner understands the words
Coverage test	Every translation, stop and the general message has a real sample: exactly 19	—
Fixture check in a real browser	The saved error texts still match what the worker reports today	Other Pyodide versions
Snapshot test	Every message template matches the reviewed copy; code in a message is one short line	—
Browser tests, real Python, mocked API	The highlight is on the right line, with ▶ and "Line N"; "Go to line" moves the cursor only when clicked; Python's message opens with a click or Enter; the highlight clears when that line is edited; hints appear in order with no answer button; a "Try it" run and a non-`passed` state never look like a pass; ENG-03 results highlight a Python error line but not an output line; phone width; reduced motion	The real API (mocked here), real screen readers
API test	The `hi/woah` answer used by the browser test is exactly what `POST /api/check` returns	—

Kind of test	What it proves	What it does not prove
Existing backend tests	No hint, or combination of a card's hints, passes its check (`test_honest_checks.py`)	—

The `print("hi")` / `print("woah")` test (the owner kept decision 5): - On "Try it", the run shows "Ran · not checked", nothing is sent to the check, and the box is not styled as a pass. - On the challenge, the real API answer shows "Check passed" with only what was verified, plus the self-check question "That part is your call." The words "setting and a surprise" appear nowhere.

Mocks: the browser tests mock every API answer, so they prove nothing about the real server. The `hi`/`woah` answer is tied to the real API by the backend test above. The ENG-03 answers are copied from `browser-checks.md` section 4. Python and the worker are real.

Deliberate-break controls (each one undone afterwards)

What I broke	Tests that failed
The line is read from the **first** `<exec>` frame instead of the last	2: `translation: name-in-function`; "line numbers come from the learner's code"
Translations ignore their unit, so number errors are taught in unit 1	1: "a translation for a later unit is not used in unit 1"
The page trusts `passed: true` instead of the result state	1: "a result is a pass only when the check says passed"
The highlight is drawn one line too low	6: NameError highlight, syntax errors, error inside a function, highlight clears, ENG-03 result, phone layout

After undoing them, the files matched the saved good copies byte for byte, and the full suite passed again (83 passed).

8. Open PRs and conflicts

Open PR	`git merge-tree` with this branch	Notes
#19 SITE-01 (`docs/site-design`)	Clean	Changes `build.md` and its own docs only
#20 BACKLOG-SYNC (`docs/backlog-sync`)	Clean	Changes `backlog.md` only; its ENG-04 row should become "in_review" (section 11)
#21 FB-01 (`feat/admin-role`)	Clean	**Trial merge:** 282 backend tests passed, 10 skipped

This PR does not depend on any of them, and none changes a file this PR changes.

9. Known limitations

- **Unit 1 hints were not written as a nudge, a bigger hint and the approach.** They are shown under those labels in their existing order. Some fit loosely; for example, `input` hint 3 is "another way is an f-string". Rewording them is content work for part 2.
- **No tagged hints yet,** so a `NameError` does not bring up the spelling hint first.
- **The editor no longer wraps long lines;** they scroll sideways inside the editor.
- **The page does not call the ENG-03 runner yet** (UI-05). Its results are only shown correctly when they arrive.
- **Python's column** is not used. The line is enough for the highlight.
- **The 15-second limit is still the app's**, so the timeout message says 15 seconds.
- **Not tested with a real screen reader,** only with accessible names, live regions and axe. Not tested on Safari or Firefox; the tests use Chromium.
- **Real error texts come from Pyodide 0.27.7.** A Pyodide upgrade needs the fixtures captured again. The fixture check test will fail and say so.

10. Live checks for you after merging and deploying

Use a student account, or `/preview` as a parent. Each takes about a minute.

- [ ] **NameError.** Open "Create your hero", card 5. Type `hero = "Kai"`, `tool = "map"`, `print(heor)` on three lines and run. Expected:
- "Python stopped", "Python does not know heor", "Line 3", "Did you mean hero?"
- line 3 marked with ▶ and tinted
- "Go to line 3" puts the cursor at the start of line 3
- [ ] **Python's message.** Click "Show Python's message". You see a short traceback for line 3, with no `/lib/python312.zip` lines.
- [ ] **Syntax error.** Open "Choose a path", card 5, and remove the `:` after an `if`. Expected: "This line needs a colon" and "On or just before line N" with that line marked.
- [ ] **The highlight clears.** Change the marked line. The marker goes, and the message stays until you run again.
- [ ] **Honest pass.** Open "Open the story", card 5. Enter `print("hi")` and `print("woah")`, then click "Run & check challenge". Expected: "Check passed" and the message ending "That part is your call." It still passes, as you decided.
- [ ] **Not a pass on "Try it".** On card 4, run any code. Expected: "Ran · not checked" in a grey box, not green.
- [ ] **Hints.** On any card 5, click the hint button three times. Expected: "A nudge", "A bigger hint", "The approach" in order, then "That is every hint" and no answer button.
- [ ] **Nothing new is sent.** In DevTools → Network, run code with an error. No request to `/api/check` or `/api/attempts` is made.

- [ ] **No CSP reports.** In DevTools → Console, no Content-Security-Policy messages appear while doing the above.
- [ ] **Phone.** Repeat the NameError check on a phone. The page does not scroll sideways.

11. Backlog rows to add (not edited here, as asked)

ID	Status	Change
ENG-04	in_review (this PR)	Part 1: translations, line highlight, hint ladder, honest result states
ENG-04b (new)	proposed	Part 2: named check rules and message IDs in content; hints tagged to errors and rules; a similar example after hint 3; "Other ways to do it" after passing; predict-first; support-data measurements. Before EDU-04 (unit 2)
EDU-xx (new, or part of ENG-04b)	proposed	Reword unit 1 hints to fit "a nudge, a bigger hint, the approach", still with no hint passing the check
UI-05	unchanged	Add: call `check-runner.js` from the lesson page for lessons with browser checks

12. Questions for the owner

1. **The split** (section 4): is ENG-04 part 2 as listed the right next stage, before unit 2?
2. **Open decision 2:** keep the friendly message first in every unit, or put Python's message first from unit 6?
3. **Open decision 3:** highlight the hint button after two failed runs, or never? (Not built.)
4. **Assumptions 1 to 3** in section 5: the timeout message in every unit; the indent message for `else` and `elif`; naming the real built-in. Are these right?
5. **Unit 1 hint wording** (section 9): reword them in part 2?

13. Next concrete action

Review the PR and run the live checks after merging. Then decide section 12, so that ENG-04 part 2 can start before unit 2 (EDU-04).